Fundamentos de DevOps

Avance de Proyecto

Equipo 5

DevOps SDF

Santiago Atahualpa Meléndez Torres

Diego Couto Manero

Fernando Samaniego Mondragón

Objetivo

Diseñar e implementar un flujo de trabajo DevOps escalable y automatizado en Amazon Web Services (AWS), integrando herramientas de control de versiones, contenedores, infraestructura como código, pipelines de CI/CD, monitoreo y seguridad, con el fin de optimizar los tiempos de entrega, mejorar la estabilidad de las aplicaciones y garantizar la seguridad de la infraestructura en la nube.

Instrucciones

Caso de análisis:

La empresa Soluciones Tecnológicas del Futuro es una organización de reciente creación y en expansión, dedicada al desarrollo de aplicaciones web para el sector financiero. Actualmente, enfrenta desafíos en la gestión y despliegue de sus soluciones en la nube. Su proceso de entrega de software es manual, lo que genera retrasos en las actualizaciones, errores durante la implementación en producción y dificultades para monitorear el rendimiento de sus aplicaciones.

Para abordar estos problemas, Soluciones Tecnológicas del Futuro ha decidido implementar una plataforma automatizada de despliegue y monitoreo en AWS, adoptando prácticas de DevOps. El objetivo de este proyecto es optimizar los tiempos de entrega, mejorar la estabilidad de sus aplicaciones y garantizar la seguridad de la infraestructura en la nube para sus clientes.

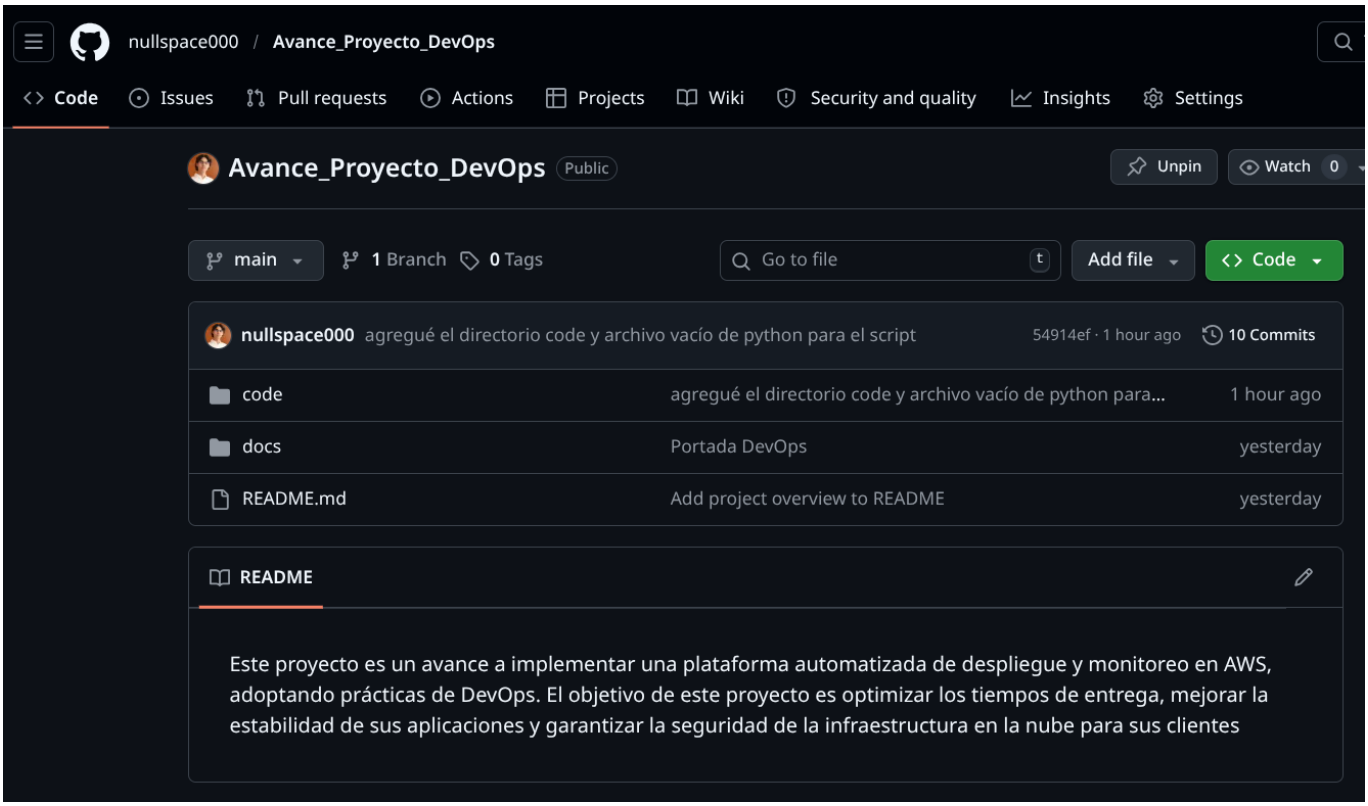
Procedimiento:

1. Elaborar una presentación que exponga los principios de DevOps

- nombre de la presentación: devops_principios.html

2. Crear un repositorio en Github

`https://github.com/nullspace000/Avance_Proyecto_DevOps.git`



3. Configurar un entorno de desarrollo en Linux

1. Instalar Ubuntu en una máquina virtual local o en una instancia AWS EC2 (solo tamaños nano, micro, small, medium o large).
2. Configurar paquetes esenciales: git, vim, docker, python3.
3. Crear y ejecutar scripts Bash para automatizar tareas.

```
# Herramientas base
sudo apt install -y git vim curl build-essential
# Python 3 y pip
sudo apt install -y python3 python3-pip python3-venv
# Docker Engine (repositorio oficial)
sudo apt install -y ca-certificates gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo \
"deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io
# Añadir tu usuario al grupo docker
sudo usermod -aG docker "$USER"
```

5. Automatizar la instalación de dependencias. Crea un archivo bootstrap_dev.sh:

```
set -euo pipefail
sudo apt update
sudo apt install -y git vim python3 python3-pip docker.io
echo "Instalación completa.
```

```
ubuntu@ip-172-31-73-186:~$ ./bootstrap_dev.sh
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 https://download.docker.com/linux/ubuntu noble InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
vim is already the newest version (2:9.1.0016-1ubuntu7.10).
python3 is already the newest version (3.12.3-0ubuntu2.1).
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Instalación completa.
```

1. Programar tareas con cron para limpieza de logs.
   Crea un script clean_logs.shque elimine o rote logs antiguos

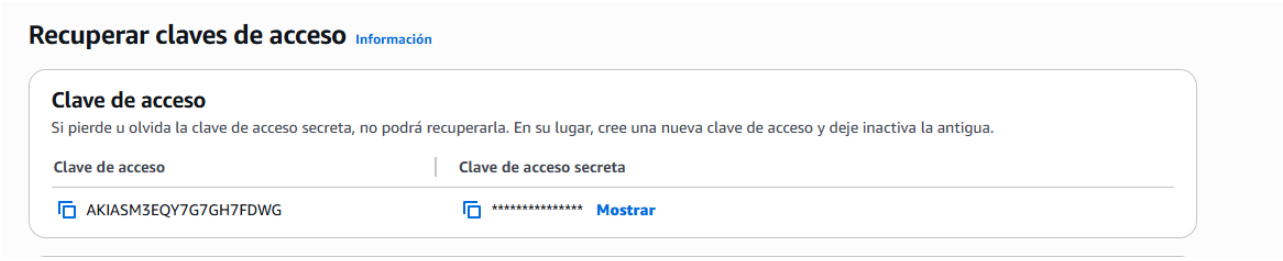
   ```
   set -euo pipefail
   LOG_DIR="/var/log/myapp"
   DAYS_TO_KEEP=7
   find "$LOG_DIR" -type f -name "*.log" -mtime +"$DAYS_TO_KEEP" -print -
   delete
   ```

Programa una tarea cron para ejecutarlo, por ejemplo diariamente a las 03:30: `crontab -e 30 3 * * * /usr/local/bin/clean_logs.sh >> /var/log/clean_logs.cron.log 2>&1`

```
ubuntu@ip-172-31-73-186:~$ crontab -e
no crontab for ubuntu - using an empty one

Select an editor.  To change later, run 'select-editor'.
  1. /bin/nano        <---- easiest
  2. /usr/bin/vim.basic
  3. /usr/bin/vim.tiny
  4. /bin/ed

Choose 1-4 [1]: 30 3 * * * /usr/local/bin/clean_logs.sh >> /var/log/clean_logs.cron.log 2>&1
Choose 1-4 [1]:
crontab: installing new crontab
```

4. Desarrollar un script en Python para automatizar tareas

1. Crear las claves de acceso IAM para poder hacer uso de los script



Configurar los datos en el EC2

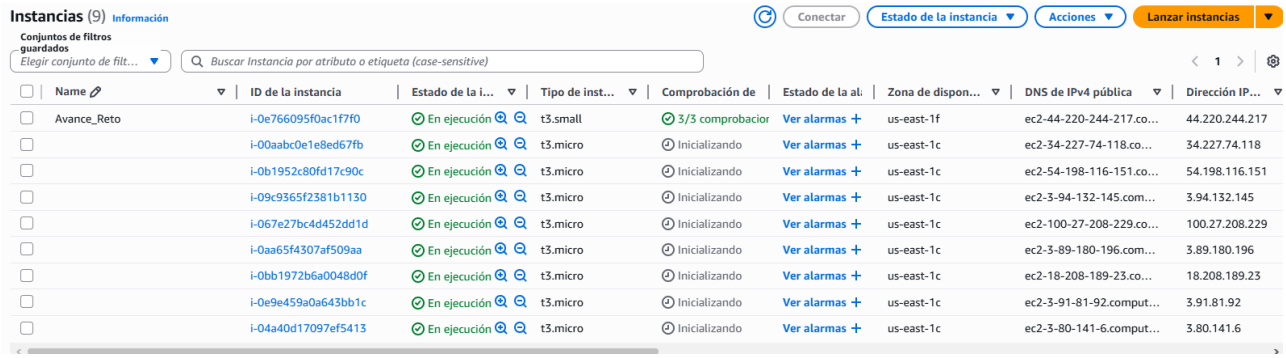
```
(devops) ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Cons

AWS Access Key ID [None]: AKIASM3EQY7G7GH7FDWG
AWS Secret Access Key [None]: j2Pf4+QrO5NzzjIFB+iYyveg/XvwWD7Q3WNTFuht
Default region name [None]: us-east-1
Default output format [None]: json
```

2. Crear un script para aprovisionar instancias EC2 (máximo 9 instancias en total, respetando los límites de Learner Lab). saber el AMI del EC2 para poder hacer la creacion

```
(devops) ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ aws ssm get-parameter \
  --name /aws/service/canonical/ubuntu/server/22.04/stable/current/amd64/hvm/ebs-gp2/ami-id \
  --query "Parameter.Value" \
  --output text \
  --region us-east-1
ami-00de3875b03809ec5
```

Correrlo

```
ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ python3  provision_ec2.py
Lanzadas 8 instancias nuevas.
```



3. Generar un reporte automático de uso de recursos.

```
ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ python3 report_s3_ec2.py
Instancias EC2 activas: 9
Buckets S3: 0
ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$
```

4. Utilizar Boto3 para interactuar con AWS.

```
ubuntu@ip-172-31-73-186:~$ source ~/.venvs/devops/bin/activate
(devops) ubuntu@ip-172-31-73-186:~$ pip install --upgrade pip
Requirement already satisfied: pip in ./.venvs/devops/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 28.9 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.0.1
(devops) ubuntu@ip-172-31-73-186:~$ pip install boto3
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ReadTimeoutError("HTTPSConnectionPool(host='pypi.org', port=443): Read timed out. (read timeout=15)")': /simple/boto3/
Collecting boto3
  Downloading boto3-1.42.85-py3-none-any.whl.metadata (6.7 kB)
Collecting botocore<1.43.0,>=1.42.85 (from boto3)
  Downloading botocore-1.42.85-py3-none-any.whl.metadata (5.9 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3)
  Downloading jmespath-1.1.0-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.17.0,>=0.16.0 (from boto3)
  Downloading s3transfer-0.16.0-py3-none-any.whl.metadata (1.7 kB)
Collecting python-dateutil<3.0.0,>=2.1 (from botocore<1.43.0,>=1.42.85->boto3)
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3!=2.2.0,<3,>=1.25.4 (from botocore<1.43.0,>=1.42.85->boto3)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting six>=1.5 (from python-dateutil<3.0.0,>=2.1->botocore<1.43.0,>=1.42.85->boto3)
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading boto3-1.42.85-py3-none-any.whl (140 kB)
Downloading botocore-1.42.85-py3-none-any.whl (14.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.8/14.8 MB 49.5 MB/s  0:00:00
Downloading jmespath-1.1.0-py3-none-any.whl (20 kB)
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Downloading s3transfer-0.16.0-py3-none-any.whl (86 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: urllib3, six, jmespath, python-dateutil, botocore, s3transfer, boto3
Successfully installed boto3-1.42.85 botocore-1.42.85 jmespath-1.1.0 python-dateutil-2.9.0.post0 s3transfer-0.16.0 six-1.17.0 urllib3-2.6.3
(devops) ubuntu@ip-172-31-73-186:~$
```

5. Listar buckets en S3 y sus objetos.

```
(devops) ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ python list_s3_inventory.py
```

5. Diseñar una plantilla CloudFormation

1. Definir infraestructura en YAML para instancias EC2 y S3, asegurando que las instancias cumplan los límites del entorno.

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: 'Stack dev: 1 bucket S3 y hasta 3 instanc

Parameters:
  InstanceType:
    Type: String
    Default: t3.micro
    AllowedValues:
      - t3.nano
      - t3.micro
      - t3.small
    Description: Tipo de instancia EC2 permitido
      You, 18 minutes ago • Creacion de archivo .yaml
  AmiId:
    Type: AWS::EC2::Image::Id
    Description: AMI de Ubuntu para la región (por ejem
```

2. Aplicar políticas de IAM solo con el rol LabRole preexistente, ya que Learner Lab no permite crear nuevos roles o grupos.

No aplica ya que no es learner lab

3. Implementar recursos en AWS mediante IaC.

```
aws cloudformation deploy \
--stack-name devops-stack \
--template-file stack-dev.yaml \
--parameter-overrides \
 AmiId=ami-00de3875b03809ec5 \
 InstanceType=t3.micro \
--region us-east-1
```

4. Desplegar y actualizar infraestructura con AWS CloudFormation deploy.

```
ubuntu@ip-172-31-73-186:~/Avance_Proyecto_DevOps/docs$ aws cloudformation deploy \
  --stack-name devops-stack \
  --template-file stack-dev.yaml \
  --parameter-overrides \
    AmiId=ami-00de3875b03809ec5 \
    InstanceType=t3.micro \
  --region us-east-1

Waiting for changeset to be created..
Waiting for stack create/update to complete
Successfully created/updated stack - devops-stack
```

6. Crear una imagen Docker para una aplicación web

1. Definir un Dockerfile con configuración de nginx o flask.

   Dockerfile:

```
# Stage 1: Build (dependency instalation)
FROM python:3.11-slim AS builder

WORKDIR /app
COPY requirements.txt .
# Instalamos dependencias en un directorio local
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Final (light image for exec)
FROM python:3.11-slim

WORKDIR /app
# Copiamos solo las dependencias instaladas y el código
COPY --from=builder /root/.local /root/.local
COPY . .

# Asegurar que el PATH incluya las librerías instaladas
ENV PATH=/root/.local/bin:$PATH

EXPOSE 5000
CMD ["python", "app.py"]
```

2. Optimizar la imagen con multi-stage builds.

   - docker compose up --build
   - tenemos stage 1 y stage 2

3. Configurar docker-compose.yml para múltiples servicios.
   docker-compose.yml:

```
version: '3.8'

services:
web:
   build: .
   ports:
      - "8080:5000"
   volumes:
      - ./app:/app  # Hot-reloading para desarrollo
   networks:
      - frontend
      - backend
   depends_on:
      - db
```

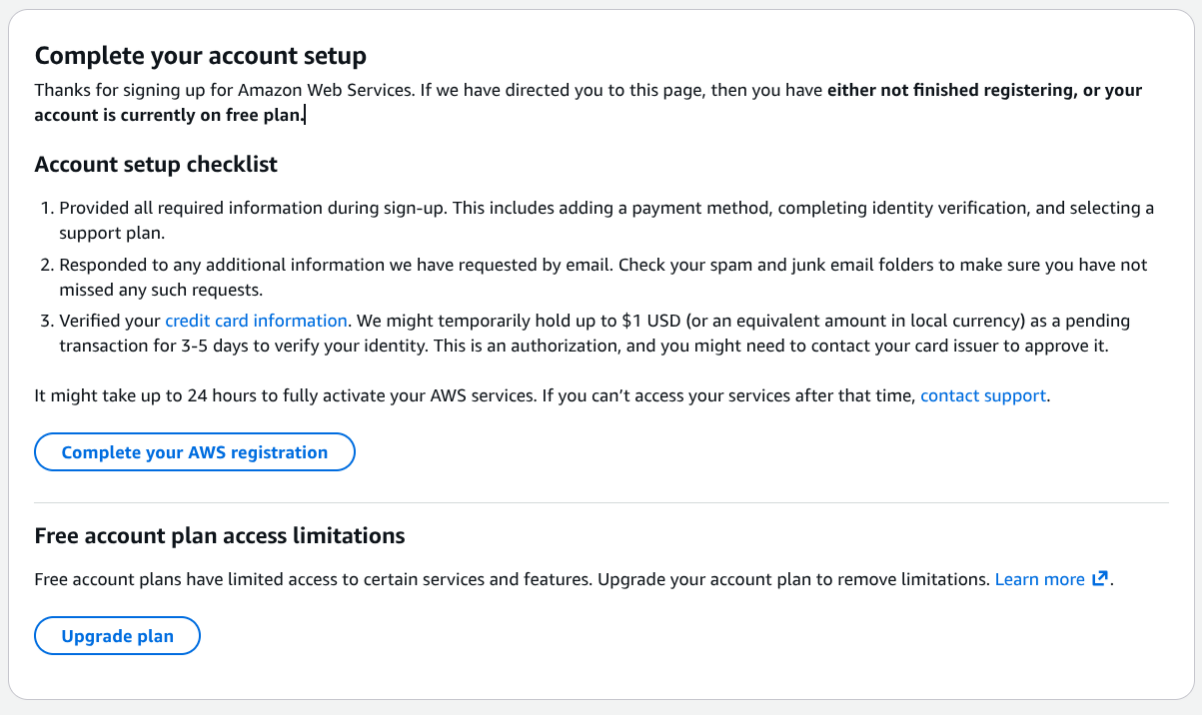
```
    environment:
       - REDIS_HOST=db

db:
   image: redis:alpine
   networks:
      - backend
   volumes:
      - redis_data:/data

networks:
frontend:
   driver: bridge
backend:
   internal: true  # Red aislada para seguridad

volumes:
redis_data:
```

4. Definir volúmenes y redes personalizadas.

```
[null@T480 avance_reto]$ sudo docker network ls
[sudo] password for null:
NETWORK ID     NAME              DRIVER    SCOPE
08b7811756f4   bridge            bridge    local
ff7efc952609   docker_backend    bridge    local
32db7fe7059d   docker_frontend   bridge    local
0c6fa98669dc   host              host      local
0e8316485271   none              null      local
```

7. Implementar un pipeline CI/CD con AWS CodeCommit

1. Configurar CodeCommit y CodeBuild para pruebas automatizadas.
    - CodeCommit y CodeBuild no están accesibles en la capa gratuita de AWS.
    - Al no tener acceso a los learner labs, no podemos continuar con esta parte del reto.



Complete your account setup

Thanks for signing up for Amazon Web Services. If we have directed you to this page, then you have **either not finished registering, or your account is currently on free plan.**

Account setup checklist

1. Provided all required information during sign-up. This includes adding a payment method, completing identity verification, and selecting a support plan.
2. Responded to any additional information we have requested by email. Check your spam and junk email folders to make sure you have not missed any such requests.
3. Verified your credit card information. We might temporarily hold up to $1 USD (or an equivalent amount in local currency) as a pending transaction for 3-5 days to verify your identity. This is an authorization, and you might need to contact your card issuer to approve it.

It might take up to 24 hours to fully activate your AWS services. If you can't access your services after that time, contact support.

Complete your AWS registration

Free account plan access limitations

Free account plans have limited access to certain services and features. Upgrade your account plan to remove limitations. Learn more.

Upgrade plan

2. Integrar CodePipeline para despliegue continuo.
3. Enviar archivos a EC2 utilizando AWS Systems Manager Session Manager.
4. Usar AWS Lambda para automatizar rollback ante fallos.